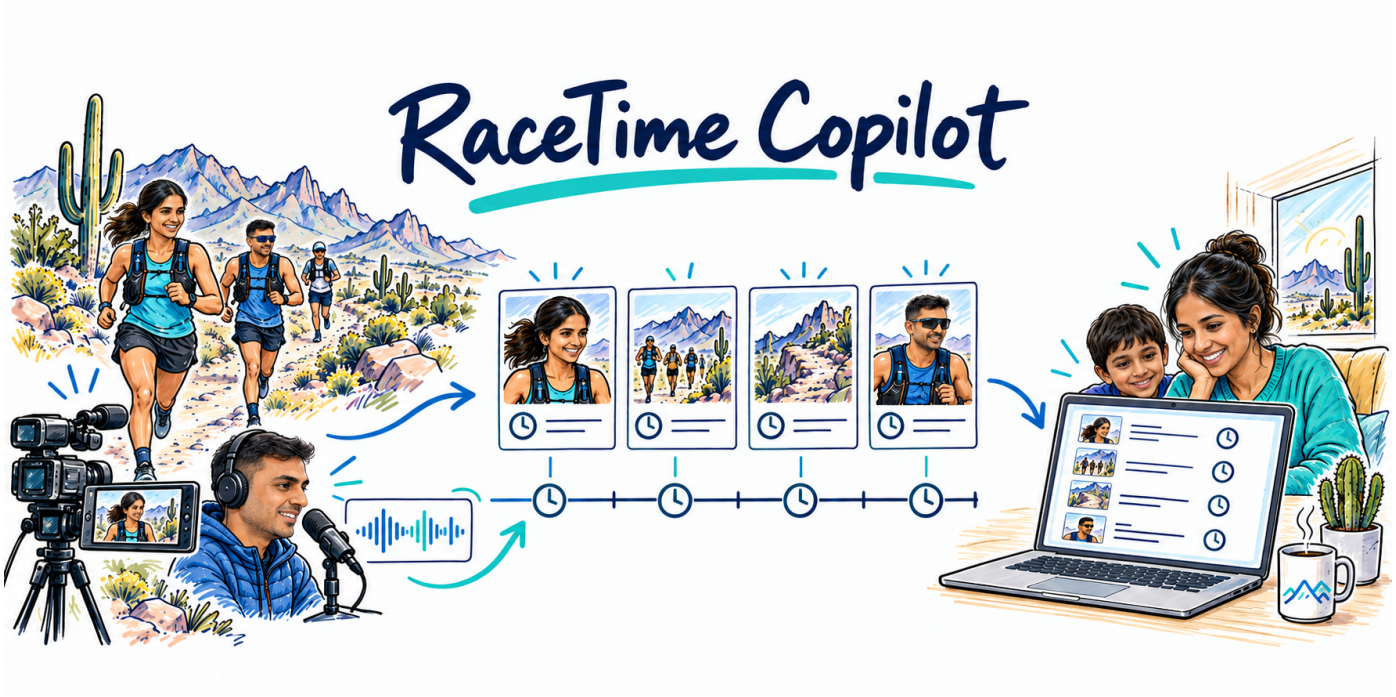
1 / THE PROBLEM

Missed 15 minutes? Catch up.

RaceTime Copilot helps you understand a chosen race interval through timestamped evidence.



The problem

Long broadcasts are hard to catch up on. Scrubbing can miss key moments or reveal future results.

The useful result

Choose a window, follow a runner, and see what the sources reported - including disagreements.

What works today

Video jobs, learned retrieval and cited recaps. A key-free evidence demo is also included. See validation limits.

2 / WHO IT HELPS

For the people following the race.

Built by Siva Babu: ultramarathoner, race organizer and observability practitioner.

Fans and families

Catch up together on a missed interval. Inspect the evidence behind a reported sighting or position.

Organizers and crews

Review imported commentary and timing context. Keep uncertain reports visible.

Why this project

A personal race-viewing problem meets systems skills: time boundaries, memory limits and useful traces.

3 / HOW IT WORKS

One question. One evidence workflow.

Save a video, choose an interval and spoiler cutoff, then ask what happened. Review the cited evidence.

Retrieve	Keep only evidence fully inside the window.	Inspect bounded video windows and rank learned evidence vectors.
Check	Generate a cited recap and check its grounding.	Exclude later evidence. Say when there is not enough evidence.
Review	Read the timestamped recap and its trace.	Pause at a durable checkpoint; approve or reject, even after a restart.

A bounded Gemini planner chooses legal actions. Coverage gaps and provider failures remain visible.

4 / TECHNOLOGY AND LEARNING

Each tool has a clear job.

Five course themes applied to one product. The detailed technology map is in docs/technology-map.md.

Week 1 / App	Streamlit, Python and persistent accounts.	Save videos, queue jobs and review results. TypeScript/Zod powers the optional evidence demo.
Week 2 / Retrieval	Gemini video, learned embeddings and citations.	Inspect a fixed interval, retrieve evidence, generate a recap and check the cited observations.
Week 3 / Workflow	LangGraph, SQLite jobs and durable review.	Bound model decisions, retry calls and resume review. Weighted LRU remains in the evidence demo.
Week 4 / Evaluation	Unit tests, AppTest, traces and human labels.	Test restart and time rules. Real-race labels and external trace quota are still required.
Week 5 / LoRA	PyTorch, Transformers, PEFT and scikit-learn.	Train, evaluate and merge a BERT-tiny adapter. This adapts the technique, not the exact Qwen3/LLaMA Factory setup.

Live capture uses FFmpeg and yt-dlp. Public deployment scaffolding is prepared; the current scope is local.

5 / EVIDENCE THAT IT WORKS

Small tests. Clear limits.

These measurements use authored synthetic data. They do not establish real-video summary quality.

38 / 38	40 / 40	Passed
17 evidence + 21 product tests	Evidence cases	Streamlit + API flows

Spoiler test	Request 00:00-15:00 of fictional Canyon Relay.	Two ridge reports disagree. The update at 15:30 stays out.
Evidence result	Naive baseline 20/40 -> workflow 40/40.	Strict time and availability rules explain the measured improvement.
Separate LoRA lab	52.5% baseline -> 70% held-out accuracy.	144 training / 40 held-out examples. The app keeps the rule router; the trained adapter is not deployed.

Real Gemini check: small uploaded video, embeddings, cited recap and durable review passed.

YouTube and large-file processing remain provider-dependent. Active live capture and race accuracy need validation.

6 / RUN IT AND EXTEND IT

A simple demo. A clear next step.

Video workspace: Python 3.11+ and a Gemini key. The optional evidence demo also needs Node.js 22.13+.

START LOCALLY

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# configure Gemini in .env
python scripts/run_product.py
```

Open http://localhost:8501

Try this first

Create a local account. Save a short video and queue a question. Open Jobs / results, check timestamps and approve or reject the recap.

For a key-free fallback, use run_demo.py and the fictional Canyon Relay scenario. Setup steps are in docs/setup.md.

Next	Validate real race clips and an active stream.	Review timestamps and claims independently. Resolve provider access and external trace quota.
Then	Pilot with race fans and organizers.	Measure time saved and evidence accuracy. Explore timing feeds, runner watchlists and multi-camera recaps.

github.com/sivalinb/racetime-copilot

Read: README -> docs/learning-guide.md -> docs/demo-guide.md. Course scope: user-provided Week 1-5 handouts.